DIVINECARE PLATFORM

Client Code Architecture & Production Readiness Review

A senior engineering assessment of security, scalability, data integrity, reliability and operational readiness - with plain-English explanations for business stakeholders.

Review date: 28 October 2026

Scope: divinecare-platform-altrixs-staging source snapshot

Executive summary

The reviewed codebase contains substantial feature development and several thoughtful safeguards. The principal concern is not whether screens and functions exist. The concern is whether the entire healthcare platform can be operated securely, consistently and recoverably in production.

The review identified risks in real-time authorization, multi-server scalability, deployment resilience, database change control, automated testing, role enforcement, file storage, scheduled jobs and failure visibility. These are system-level responsibilities that remain after feature development is considered complete.

Area	Observed condition	Risk
Real-time messaging	Unauthenticated socket design if enabled	Critical
Scalability	Process-local events and single-server coupling	High
Database	Prisma plus raw SQL and manual baseline history	High
Testing	No real database/integrations in CI; tests omitted	High
Authorization	Policy spread across hundreds of API handlers	High
Operations	Host cron, local files and limited failure visibility	High

PLAIN-ENGLISH EXPLANATION

The product may look nearly finished from the outside, but the behind-the-scenes safety systems still need specialist work. It is similar to finishing the rooms of a hospital before validating the fire system, backup power, access controls and emergency procedures.

Recommended engagement

Stop open-ended feature development if that is the client's preference, but retain an accountable senior engineering team for architecture hardening, security validation, database reconciliation, integration testing and production release certification.

Key technical findings

1 Real-time messaging is not safely authenticated if enabled

CRITICAL

The Socket.IO server accepts connections from any origin, trusts a user object supplied by the browser and permits a caller to join a room by supplying a thread ID. The server does not independently verify the session, the claimed identity or membership of the requested conversation. The feature is disabled by default because the current Next.js production server arrangement cannot initialize it correctly, but switching it on without redesign would create an authorization exposure.

Business impact: Potential unauthorized exposure of message events and presence data if the configuration is enabled without additional controls.

PLAIN-ENGLISH EXPLANATION	A person should only be able to enter a chat room after the server checks their identity and confirms that the room belongs to them. Here, the proposed socket design accepts the name and room number supplied by the browser. It is like letting a visitor write their own ID badge and choose which consultation room to enter.

Code evidence: src/lib/socket/server.ts lines 25-96; src/features/messages/hooks/useSocketChat.ts lines 44-72; src/app/api/socket/io/route.ts lines 6-27.

2 Real-time delivery is tied to one Node.js process

HIGH

Message events are distributed through an in-memory EventEmitter stored on globalThis. This event bus is not shared between application instances and is cleared by restarts. The optional Pusher publication is fire-and-forget and silently suppresses delivery failures. This architecture cannot guarantee consistent event delivery when the application is scaled horizontally or restarted.

Business impact: Missed or inconsistent message notifications, especially during scaling, deployment or transient network failure.

PLAIN-ENGLISH EXPLANATION	Each server keeps its own private notice board. If two users are connected to different servers, a notice placed on one board may never appear on the other. Restarting the server clears the board completely.

Code evidence: src/lib/messages/realtime.ts lines 35-101 and 72-82.

3 The current environment has a single-server failure domain

HIGH

The staging runbook documents one EC2 application instance, PM2 restart deployment, locally configured cron jobs and filesystem-based uploads. These components share the availability of the same host. A host failure can affect the portal, scheduled integrations, message streams and document access together.

Business impact: Extended outage and delayed clinical or billing integrations during instance failure or maintenance.

PLAIN-ENGLISH EXPLANATION	Too many essential functions depend on one machine. If that machine stops, the website, scheduled work and some file access can stop at the same time. A production healthcare system needs another route when one machine fails.

Code evidence: DEPLOY.md lines 3-15 and 51-69; deploy/crontab.staging.

4 Database change control has two sources of truth

HIGH

The codebase contains a 9,244-line Prisma schema with approximately 320 models plus more than 100 SQL migration files. The deployment documentation confirms that some database structures exist only in SQL and warns that Prisma db push could remove them. It also records that earlier staging changes were hand-applied and later baselined. The migration script has no concurrency lock.

Business impact: Schema drift, failed deployments, difficult rollback and possible loss or misinterpretation of clinical and billing data.

PLAIN-ENGLISH EXPLANATION	The application's map of the database does not show every road. Some changes are recorded in one system and other changes in another. If an engineer follows only one map, valid information can be removed or the database can end up in a state that the code does not expect.

Code evidence: prisma/schema.prisma; DEPLOY.md lines 29-36, 75-89 and 122-141.

5 CI does not exercise the actual database or external integrations

HIGH

The CI workflow explicitly states that its build and tests never connect to a database or integration. Passing CI therefore does not validate migrations, PostgreSQL behavior, DrFirst, Waystar, Stripe, AWS, Chime, HealthLake, email or SMS as a complete deployed workflow.

Business impact: Integration defects can remain undiscovered until staging or production.

PLAIN-ENGLISH EXPLANATION	The automated checks test many individual parts on a workbench, but they do not start the complete machine with its real connections. A green check mark does not prove that a prescription, claim, payment or clinical workflow will complete in production.

Code evidence: .github/workflows/ci.yml lines 19-27 and 46-63.

6 Important test files are outside the normal CI test command

HIGH

The npm test script names test files individually rather than discovering the test suite automatically. Thirteen top-level tests are omitted, including security hardening, role separation, registration persistence, treatment-team access, onboarding verification and Valant repair integration tests.

Business impact: Security or persistence regressions may not block a release.

PLAIN-ENGLISH EXPLANATION	Some safety inspections have been written, but the normal automated process does not actually run them. Their existence can make the test coverage look stronger than the protection provided by each release.

Code evidence: package.json script test; omitted files include security-hardening-oct18.test.ts, role-separation-billing-office.test.ts and registration-persistence.test.ts.

7 Known lint failures are allowed to pass

HIGH

The CI lint step uses continue-on-error and is described as non-blocking until existing errors are fixed. A release can therefore receive a successful CI result while still containing known quality violations.

Business impact: Known defects and unsafe patterns can accumulate without a release gate.

PLAIN-ENGLISH EXPLANATION	The inspection process finds problems but is instructed not to stop the release. Before production, critical inspections should be mandatory, not advisory.

Code evidence: .github/workflows/ci.yml lines 55-57.

8 Authorization rules are distributed across a very large API surface

HIGH

The application contains approximately 799 API route handlers and about 1,796 direct role-string occurrences. Authorization is implemented through central gates in some routes, patient-specific handlers in others and local role sets elsewhere. This distribution makes it difficult to prove that every read and write operation applies the same tenant, patient and role policy.

Business impact: Permission drift, inconsistent tenant isolation and expensive access-control review.

PLAIN-ENGLISH EXPLANATION	There are hundreds of doors, and the rules for who may enter are written beside many individual doors instead of being controlled by one consistent security desk. Over time, one door can easily receive a different rule or miss an update.

Code evidence: src/app/api route inventory; local role example in src/app/api/medical/payer-settings/route.ts.

9 Some infrastructure failures are converted into normal-looking results

HIGH

Several workflows suppress exceptions or return fallbacks. In the medication route, failure to load fill events is ignored and the response can use normal-looking adherence defaults such as dispensed and ok. Pusher failures are silently discarded. Location database failure can return an empty fallback list. Operational failure should be represented as unknown or unavailable, not as successful clinical state.

Business impact: Users may make decisions using stale or incorrectly reassuring information.

PLAIN-ENGLISH EXPLANATION	If the system cannot retrieve an answer, it sometimes shows a normal answer instead of warning that the information is unavailable. In healthcare and billing, 'we could not check' must never look like 'everything is fine.'

Code evidence: src/app/api/clinical/medications/route.ts lines 101-115 and 143-146; src/lib/messages/realtime.ts lines 78-82; src/app/api/locations/route.ts lines 42-56.

10 Patient API CORS configuration fails open

HIGH

The shared patient response layer defaults Access-Control-Allow-Origin to * when PATIENT_CORS_ORIGIN is absent. This behavior is used across dozens of patient API files. While native mobile bearer-token APIs may require broad access, a wildcard is not a safe default for browser-accessible healthcare endpoints. Production should validate exact origins at startup and refuse unsafe configuration.

Business impact: Unnecessary browser exposure if an environment variable is omitted or misconfigured.

PLAIN-ENGLISH EXPLANATION	If an important website access list is missing, the application currently changes the rule to 'allow every website.' A safer approach is to stop and request the correct list.

Code evidence: src/patient/infrastructure/http/response.ts lines 5-13; docs/production-cutover-plan.md requires exact origins.

11 Private documents are coupled to filesystem storage

HIGH

Clinical documents, consents, credentials, intake documents, resources and avatars are written beneath UPLOADS_DIR. This requires a correctly mounted shared filesystem for multi-server operation and coordinated backup between database records and file contents. The production plan proposes EFS, but deployment, encryption, backup, malware scanning and restoration must be implemented and proven.

Business impact: Missing files, inconsistent restore, scaling constraints and compliance exposure.

PLAIN-ENGLISH EXPLANATION	The database remembers where a document should be, while the document itself lives on a disk. If the disk, mount or backup is wrong, the record may remain but the actual clinical document may be missing.

Code evidence: src/lib/storage/privateUploads.ts; document and consent storage modules; docs/production-cutover-plan.md.

12 Critical scheduled work depends on machine-level cron

HIGH

DrFirst synchronization, Waystar retrieval, consent reminders, eligibility checks and other jobs are configured through /etc/cron.d on the application host. The configuration contains an unresolved TODO to verify duplicate jobs and server log paths. The jobs use different locking approaches and there is no demonstrated centralized retry, dead-letter or alerting system.

Business impact: Delayed prescriptions, claims, eligibility results, reminders or synchronizations without immediate detection.

PLAIN-ENGLISH EXPLANATION	Important background work depends on one computer's alarm clock. If the computer is replaced, misconfigured or unavailable, the alarm may not ring and the business may not immediately know which tasks were missed.

Code evidence: deploy/crontab.staging lines 1-55.

13 Test quantity overstates executed workflow coverage

MEDIUM

At least 132 of 221 top-level test files read source files or check textual patterns. These guardrails can be useful, but they do not render the interface, execute a browser journey, connect to PostgreSQL or prove that authorization and multi-step clinical workflows behave correctly at runtime.

Business impact: Stakeholders may interpret a high test count as stronger production assurance than it provides.

PLAIN-ENGLISH EXPLANATION

Many tests check that the instruction label exists, rather than asking a person to follow the instruction and confirming the result. Both types are useful, but they provide different levels of confidence.

Code evidence: tests/layout-static-guards-business.test.ts explicitly states that its checks do not render anything.

14 The modular monolith has outgrown informal boundaries

MEDIUM

The branch contains approximately 583,000 JavaScript and TypeScript lines, 799 API handlers, 320 data models and multiple page components between 2,000 and 3,300 lines. A monolith can be appropriate, but at this scale it needs enforced module ownership, dependency rules, smaller use-case services and documented contracts.

Business impact: Slower changes, larger regression surface, difficult onboarding and high dependence on a few people.

PLAIN-ENGLISH EXPLANATION

The application has become a very large building with many departments sharing walls and wiring. A small repair in one department can unexpectedly affect another unless the boundaries are deliberately engineered and tested.

Code evidence: Examples include appointments/page.tsx, clients/[id]/page.tsx, notes/page.tsx, telehealth/page.tsx and communications/page.tsx.

What the client should understand

> *Feature completion is not the same as production readiness. The remaining work is less visible than building screens, but it is the work that protects patient information, prevents data loss and keeps the service available when something fails.*

Why continued senior engineering involvement is justified

- Security ownership: redesign socket authentication, review patient API exposure and produce an auditable endpoint-permission matrix.
- Data ownership: reconcile Prisma and SQL, test every migration against a realistic database copy and introduce deployment locking.
- Reliability ownership: remove single-host dependencies, establish shared storage and introduce durable event and job processing.
- Quality ownership: run all tests automatically, make lint and security gates mandatory and add database-backed integration tests.
- Operational ownership: add centralized metrics, alerts, job monitoring, integration dashboards and tested recovery procedures.

Recommended stabilization engagement

Phase	Workstream	Outcome
1	Security and access control	Inventory all APIs; verify tenant and patient scope; replace client-trusted socket identity; lock CORS configuration; test protected-record controls.
2	Database assurance	Reconcile Prisma with raw SQL; verify the migration ledger; test on a production-like clone; implement locking and recovery drills.
3	Reliability engineering	Introduce a durable event/job layer, shared encrypted document storage, centralized monitoring, retry policies and failure alerts.
4	Release certification	Execute real PostgreSQL integration tests and end-to-end clinical, billing, prescribing, communication and recovery scenarios.

Suggested client-facing statement

> *We recognize the significant feature development already completed. We are not proposing unnecessary additional screens. Our review shows that the platform has reached a level of complexity where production safety depends on architecture, security and operational engineering. The remaining risks include real-time authorization, database change control, multi-server reliability, scheduled integration processing, file durability and test coverage against real infrastructure. We propose moving from open-ended development to a defined stabilization and production-assurance engagement with measurable acceptance criteria.*

Proposed acceptance evidence

- Signed API authorization matrix covering tenant, role and patient scope.
- Database migration rehearsal with before-and-after reconciliation and recovery evidence.
- All test files discovered automatically and mandatory quality gates passing.
- Database-backed integration suite for critical clinical and billing paths.
- Authenticated, multi-instance-safe messaging and background processing.
- Documented recovery targets, monitoring dashboards and actionable alerts.

Review scope and limitations

This report is based on static inspection of the supplied divinecare-platform-altrixs-staging source snapshot. No production credentials, production database, live infrastructure or external vendor environments were used. Findings describe code and documented architecture present in the snapshot; they do not claim that every planned production control is absent from infrastructure outside the repository.

Security findings should be confirmed through controlled runtime testing. The objective is risk reduction and release assurance, not criticism of the amount of development completed.

Evidence index

Area	Primary repository evidence
Socket authorization	src/lib/socket/server.ts; src/features/messages/hooks/useSocketChat.ts
Message scaling	src/lib/messages/realtime.ts
Deployment	DEPLOY.md; deploy/codebuild; deploy/crontab.staging
Database	prisma/schema.prisma; prisma/migrations; scripts/migrate.sh
CI and testing	.github/workflows/ci.yml; package.json; tests/
Patient CORS	src/patient/infrastructure/http/response.ts
Private uploads	src/lib/storage/privateUploads.ts and document stores
Medication fallback	src/app/api/clinical/medications/route.ts

Final conclusion

The client branch demonstrates significant feature progress. It also demonstrates why accountable senior engineering remains necessary: the highest remaining risks are cross-cutting risks that individual feature completion does not resolve. The appropriate next step is a fixed, evidence-driven stabilization and production-assurance engagement.